

Priority Scheduling Non-Preemptive:

```
#include <stdio.h>

#include <stdbool.h>

int main() {
    int n;
    printf("Nitya R Swadi, 1BM24CS361\n");

    printf("Enter number of processes: ");
    scanf("%d", &n);

    int arrival_time[n], burst_time[n], priority[n];
    int completion_time[n], turnaround_time[n], waiting_time[n];
    bool completed[n];

    for (int i = 0; i < n; i++) {
        printf("\nProcess P%d\n", i + 1);

        printf("Enter Arrival Time: ");
        scanf("%d", &arrival_time[i]);

        printf("Enter Burst Time: ");
        scanf("%d", &burst_time[i]);

        printf("Enter Priority: ");
        scanf("%d", &priority[i]);

        completed[i] = false;
    }
```

```

int current_time = 0;
int completed_count = 0;

while (completed_count < n) {

    int highest_priority = 9999;
    int selected_process = -1;

    for (int i = 0; i < n; i++) {
        if (arrival_time[i] <= current_time && !completed[i]) {

            if (priority[i] < highest_priority) {
                highest_priority = priority[i];
                selected_process = i;
            }
        }
    }

    if (selected_process == -1) {
        current_time++;
    }
    else {
        int start_time = current_time;

        completion_time[selected_process] =
            start_time + burst_time[selected_process];

        turnaround_time[selected_process] =
            completion_time[selected_process] -
            arrival_time[selected_process];
    }
}

```

```

    waiting_time[selected_process] =
        turnaround_time[selected_process] -
        burst_time[selected_process];

    current_time = completion_time[selected_process];

    completed[selected_process] = true;
    completed_count++;
}
}

float total_wt = 0, total_tat = 0;

printf("\nProcess\tAT\tBT\tPriority\tWT\tTAT\n");

for (int i = 0; i < n; i++) {
    printf("P%d\t%d\t%d\t%d\t\t%d\t\t%d\n",
        i + 1,
        arrival_time[i],
        burst_time[i],
        priority[i],
        waiting_time[i],
        turnaround_time[i]);

    total_wt += waiting_time[i];
    total_tat += turnaround_time[i];
}

printf("\nAverage Waiting Time = %.2f", total_wt / n);
printf("\nAverage Turnaround Time = %.2f\n", total_tat / n);

```

```
    return 0;
}
```

OUTPUT:

```
C:\Users\Admin\Desktop\Nitya > Nitya R Swadi, 1BM24CS361
Enter number of processes: 3

Process P1
Enter Arrival Time: 0
Enter Burst Time: 2
Enter Priority: 4

Process P2
Enter Arrival Time: 1
Enter Burst Time: 5
Enter Priority: 1

Process P3
Enter Arrival Time: 1
Enter Burst Time: 3
Enter Priority: 2

Process AT    BT    Priority    WT    TAT
P1      0      2      4        0      2
P2      1      5      1        1      6
P3      1      3      2        6      9

Average Waiting Time = 2.33
Average Turnaround Time = 5.67

Process returned 0 (0x0)   execution time : 31.275 s
Press any key to continue.
```

Priority Scheduling Preemptive:

```
#include <stdio.h>
```

```
int main() {
```

```
int n;
```

```
printf("Nitya R Swadi, 1BM24CS361");
```

```
printf("Enter number of processes: ");
```

```
scanf("%d", &n);
```

```
int arrival_time[n], burst_time[n], priority[n];
```

```
int remaining_time[n];
```

```
int completion_time[n], turnaround_time[n], waiting_time[n];
```

```
for(int i = 0; i < n; i++) {
```

```
    printf("\nProcess P%d\n", i + 1);
```

```
    printf("Enter Arrival Time: ");
```

```
    scanf("%d", &arrival_time[i]);
```

```
    printf("Enter Burst Time: ");
```

```
    scanf("%d", &burst_time[i]);
```

```
    printf("Enter Priority: ");
```

```
    scanf("%d", &priority[i]);
```

```
    remaining_time[i] = burst_time[i];
```

```
}
```

```
int current_time = 0;
```

```
int completed = 0;
```

```
while(completed < n) {
```

```

int highest_priority = 9999;
int selected_process = -1;

for(int i = 0; i < n; i++) {

    if(arrival_time[i] <= current_time && remaining_time[i] > 0) {

        if(priority[i] < highest_priority) {
            highest_priority = priority[i];
            selected_process = i;
        }
    }
}

if(selected_process == -1) {
    current_time++;
}
else {

    remaining_time[selected_process]--;

    current_time++;

    if(remaining_time[selected_process] == 0) {

        completion_time[selected_process] = current_time;
        turnaround_time[selected_process] =
            completion_time[selected_process] - arrival_time[selected_process];

        waiting_time[selected_process] =
            turnaround_time[selected_process] - burst_time[selected_process];
    }
}

```

```

        completed++;
    }
}
}

float total_wt = 0, total_tat = 0;

printf("\nProcess\tAT\tBT\tPriority\tWT\tTAT\n");

for(int i = 0; i < n; i++) {

    printf("P%d\t%d\t%d\t%d\t\t%d\t%d\n",
        i + 1,
        arrival_time[i],
        burst_time[i],
        priority[i],
        waiting_time[i],
        turnaround_time[i]);

    total_wt += waiting_time[i];
    total_tat += turnaround_time[i];
}

printf("\nAverage Waiting Time = %.2f", total_wt / n);
printf("\nAverage Turnaround Time = %.2f\n", total_tat / n);

return 0;
}

```

OUTPUT:

```
C:\Users\Admin\Desktop\Nitya  X + v
Nitya R Swadi, 1BM24CS361
Enter number of processes: 3

Process P1
Enter Arrival Time: 0
Enter Burst Time: 3
Enter Priority: 5

Process P2
Enter Arrival Time: 1
Enter Burst Time: 5
Enter Priority: 1

Process P3
Enter Arrival Time: 2
Enter Burst Time: 3
Enter Priority: 2

Process AT      BT      Priority      WT      TAT
P1      0      3      5      8      11
P2      1      5      1      0      5
P3      2      3      2      4      7

Average Waiting Time = 4.00
Average Turnaround Time = 7.67

Process returned 0 (0x0)  execution time : 39.658 s
Press any key to continue.
```

Round Robin Scheduling:

```
#include <stdio.h>
```

```
int main() {
```

```
    int n, tq;
```

```
    printf("Nitya R Swadi, 1BM24CS361\n");
```

```
    printf("Enter number of processes: ");
```

```
    scanf("%d", &n);
```



```

printf("Enter Time Quantum: ");
scanf("%d", &tq);

int at[n], bt[n], rt[n];
int wt[n], tat[n];
int completed = 0, current_time = 0;

for(int i = 0; i < n; i++) {
    printf("\nProcess P%d\n", i + 1);

    printf("Enter Arrival Time: ");
    scanf("%d", &at[i]);

    printf("Enter Burst Time: ");
    scanf("%d", &bt[i]);

    rt[i] = bt[i]; // remaining time
}

while(completed < n) {

    int done = 1;

    for(int i = 0; i < n; i++) {

        if(at[i] <= current_time && rt[i] > 0) {

            done = 0;

            if(rt[i] > tq) {

```

```

        current_time += tq;
        rt[i] -= tq;

    } else {

        current_time += rt[i];

        tat[i] = current_time - at[i];
        wt[i] = tat[i] - bt[i];

        rt[i] = 0;
        completed++;
    }
}

if(done)
    current_time++;
}

float total_wt = 0, total_tat = 0;

printf("\nProcess\tAT\tBT\tWT\tTAT\n");

for(int i = 0; i < n; i++) {

    printf("P%d\t%d\t%d\t%d\t%d\n",
        i + 1, at[i], bt[i], wt[i], tat[i]);

    total_wt += wt[i];
    total_tat += tat[i];
}

```

```

}

printf("\nAverage Waiting Time = %.2f", total_wt / n);
printf("\nAverage Turnaround Time = %.2f\n", total_tat / n);

return 0;
}

```

OUTPUT:

```

C:\Users\Admin\Desktop\Nitya R Swadi, 1BM24CS361
Nitya R Swadi, 1BM24CS361
Enter number of processes: 3
Enter Time Quantum: 2

Process P1
Enter Arrival Time: 0
Enter Burst Time: 4

Process P2
Enter Arrival Time: 0
Enter Burst Time: 3

Process P3
Enter Arrival Time: 1
Enter Burst Time: 2

Process AT    BT    WT    TAT
P1      0      4      4      8
P2      0      3      6      9
P3      1      2      3      5

Average Waiting Time = 4.33
Average Turnaround Time = 7.33

Process returned 0 (0x0)   execution time : 14.735 s
Press any key to continue.

```